Future, CompletableFuture and Callable

"Concept && Coding" YT Video Notes

```
public static void main(String args[]) {

    ThreadPoolExecutor poolExecutor = new ThreadPoolExecutor(1, 1, TimeUnit.HOURS, new ArrayBlockingQueue<Runn
        Executors.defaultThreadFactory(), new ThreadPoolExecutor.AbortPolicy());

//new thread will be created here: lets say thread1
Future<?> futureObj = poolExecutor.submit(() -> {
    System.out.println("Doing the task, which thread will execute");
});

//Now, what if caller want to know the status of the thread1. Whether its completed or failed etc.
}
```

New thread will be created here: lets say thread1

Now, what if caller want to know the status of the thread1. Whether its completed or failed etc.

Future:

- Interface which Represents the result of the Async task.
- Means, it allow you to check if:
  - Computation is complete
  - Get the result
  - Take care of exception if any etc.

```
public static void main(String args[]) {

    ThreadPoolExecutor poolExecutor = new ThreadPoolExecutor(1, 1, TimeUnit.HOURS, new ArrayBlockingQueue<Runn
        Executors.defaultThreadFactory(), new ThreadPoolExecutor.AbortPolicy());

//new thread will be created here: lets say thread1
Future<?> futureObj = poolExecutor.submit(() -> {
    System.out.println("Doing the task, which thread will execute");
});

//Caller is checking the status of the thread1 is created
System.out.println(futureObj.isDone());
}
```

S.No.	Method Available in Future Interface	Purpose
1.	boolean cancel(boolean mayInterruptIfRunning)	• Attempts to cancel the execution of the task. • Returns false, if task can not be cancelled (typically bcze task already completed); returns true otherwise.
2.	boolean isCancelled()	• Returns true, if task was cancelled before it get completed.
3.	boolean isDone()	• Returns true if this task completed. • Completion may be due to normal termination, an exception, or cancellation -- in all of these cases, this method will return true
4.	V get()	• Wait if required, for the completion of the task. • After task completed, retrieve the result if available.
5.	V get(long timeout, TimeUnit unit)	• Wait if required, for at most the given timeout period. • Throws 'TimeoutException' if timeout period finished and task is not yet completed.

Example:

```
public static void main(String args[]) {

    ThreadPoolExecutor poolExecutor = new ThreadPoolExecutor(1, 1, TimeUnit.HOURS, new ArrayBlockingQueue<Runn
        Executors.defaultThreadFactory(), new ThreadPoolExecutor.AbortPolicy());

    Future<?> futureObj = poolExecutor.submit(() -> {
        try {
            Thread.sleep(7000);
            System.out.println("Doing the task, which thread will execute");
        } catch (InterpTion e) {
        }
    });

    System.out.println("Is Done: " + futureObj.isDone());
    try {
        futureObj.get(1, TimeUnit.SECONDS);
    } catch (TimeoutException e) {
        System.out.println("Time-Elapsed Happened");
    }
    catch (InterpTion e) {
    }

    try {
        futureObj.get();
    } catch (InterpTion e) {
    }

    System.out.println("Is Done: " + futureObj.isDone());
    System.out.println("Is Cancelled: " + futureObj.isCancelled());
}
```

Callable:

- Callable represents the task which need to be executed just like Runnable.
- But difference is:
  - Runnable do not have any Return type.
  - Callable has the capability to return the value.

Runnable Interface:	Callable Interface:

`@FunctionalInterface`
`public interface Runnable {`
`  public abstract void run();`
`}`
	`@FunctionalInterface`
	`public interface Callable<V> {`
	`  V call() throws Exception;`
	`}`

Example:

```
public static void main(String args[]) {

    ThreadPoolExecutor poolExecutor = new ThreadPoolExecutor(corePoolSize, maximumPoolSize, keepAliveTime, T
        new ArrayBlockingQueue<Runnable>(workQueue), Executors.defaultThreadFactory(), new ThreadPoolExecutor.AbortPolicy());

    //Runnable:
    Future<?> futureObj1 = poolExecutor.submit(() -> {
        System.out.println("Hello with Runnable");
    });

    try {
        Object object = futureObj1.get();
        System.out.println(object == null);
    } catch (InterpTion e) {
    }

    //Callable:
    ListInteger output = new ArrayList<>();
    FutureListInteger> futureObj2 = poolExecutor.submit(() -> {
        output.add(100);
        System.out.println("Hello with Runnable and Return object");
        return 0;
    });

    try {
        ListInteger outputFromFutureObj2 = futureObj2.get();
        System.out.println(outputFromFutureObj2.get(0));
    } catch (InterpTion e) {
    }

    //Callable2:
    FutureListInteger> futureObj3 = poolExecutor.submit(() -> {
        System.out.println("Hello with Callable");
        ListInteger listObj = new ArrayList<>();
        listObj.add(200);
        return listObj;
    });

    try {
        ListInteger outputFromFutureObj3 = futureObj3.get();
        System.out.println(outputFromFutureObj3.get(0));
    } catch (InterpTion e) {
    }
}
```

CompletableFuture:

- Introduced in Java8
- To help in async programming.
- We can considered it as an advanced version of Future provides additional capability like chaining.

How to use this:

1. CompletableFuture.supplyAsync:

   public static<T> CompletableFuture<T> supplyAsync(Supplier<T> supplier)

   public static<T> CompletableFuture<T> supplyAsync(Supplier<T> supplier, Executor executor)

   - *supplyAsync* method initiates an Async operation.
   - 'supplier' is executed asynchronously in a separate thread.
   - If we want more control on Threads, we can pass Executor in the method.
   - By default its uses, shared **Fork-Join Pool** executor. it dynamically adjust its pool size based on processors.

```
public class CompletableFutureExample {

    public static void main(String args[]) {

        try {
            ThreadPoolExecutor poolExecutor = new ThreadPoolExecutor(corePoolSize, maximumPoolSize, keepAliveTime,
                TimeUnit.HOURS, new ArrayBlockingQueue<Runnable>(workQueue),
                Executors.defaultThreadFactory(), new ThreadPoolExecutor.AbortPolicy());

            CompletableFuture<String> asyncTask = CompletableFuture.supplyAsync(() -> {
                //Time which thread need to execute
                return "Concept and Ty";
            }, poolExecutor);

            System.out.println(asyncTask.get());
        } catch (InterpTion e) {
        }
    }
}
```

2. *thenApply* & *thenApplyAsync*:

   - Apply a function to the result of previous Async computation.
   - Return a new *CompletableFuture* object.

```
CompletableFuture<String> asyncTask = CompletableFuture.supplyAsync(() -> {
    //Time which thread need to execute
    return "Concept and Ty";
}, poolExecutor).thenApply(s1->{String val = "";
    //Functionality which can work on the result of previous async task
    return val + "Coding";
});
```

'thenApply' method:

- Its a Synchronous execution.
- Means, it uses same thread which completed the previous Async task.

'thenApplyAsync' method:

- Its a Asynchronous execution.
- Means, it uses different thread (*from fork-join' pool, if we do not provide the executor in the method*), to complete this function.
- If Multiple '*thenApplyAsync*' is used, ordering can not be guarantee, they will run concurrently.

```
public class ThenAsyncExecutionDemo {

    public static void main(String[] args) {

        ThreadPoolExecutor poolExecutor = new ThreadPoolExecutor(corePoolSize, maximumPoolSize, keepAliveTime,
            TimeUnit.HOURS, new ArrayBlockingQueue<Runnable>(workQueue),
            Executors.defaultThreadFactory(), new ThreadPoolExecutor.AbortPolicy());

        ThreadPoolExecutor.shutdown();
        System.out.println("Main thread completed");
        System.out.println("Main thread is alive: " + Thread.currentThread().isAlive());
        System.out.println("Main thread name: " + Thread.currentThread().getName());
        System.out.println("Main thread priority: " + Thread.currentThread().getPriority());
        System.out.flush();

        System.out.println("Waiting for 10 sec...");
        Thread.currentThread().getSleep(10000);
    } catch (InterpTion e) {
    }
}
}
```

Output:

```
Main thread completed
Main thread is alive: true
Main thread name: main
Main thread priority: 5
Waiting for 10 sec...
Thread Name which runs 'supplyAsync': pool-1-thread-1
Thread Name for which runs 'thenApply': ForkJoinPool.commonPool-worker-0
```

3. *thenCompose and thenComposeAsync:*

   - *Chain together dependent Async operations.*
   - *Means when next Async operation depends on the result of the previous Async one. We can tied them together.*
   - *For async tasks, we can bring some Ordering using this.*

```
CompletableFuture<String> asyncTask1 = CompletableFuture
        .supplyAsync(() -> {
            System.out.println("Thread Name which runs 'supplyAsync': " + Thread.currentThread().getName());
            return "Concept and Ty";
        }, poolExecutor).
        thenCompose(s1->{System.out.println("Thread Name which runs 'thenCompose': " + Thread.currentThread().getName());
            return val + "Coding";
        });
```

4. *thenAccept and thenAcceptAsync:*

   - *Generally end stage, in the chain of Async operations*
   - *It does not return anything.*

```
CompletableFuture<Void> asyncTask1 = CompletableFuture
        .supplyAsync(() -> {
            System.out.println("Thread Name which runs 'supplyAsync': " + Thread.currentThread().getName());
            return "Concept and Ty";
        }, poolExecutor).
        thenAccept(s1->{System.out.println("All stages completed");});
```

5. *thenCombine and thenCombineAsync:*

   - *Used to combine the result of 2 Comparable Future.*

```
CompletableFuture<Integer> asyncTask1 = CompletableFuture
        .supplyAsync(() -> {
            return 10;
        }, poolExecutor);
CompletableFuture<Integer> asyncTask2 = CompletableFuture
        .supplyAsync(() -> {
            return 5;
        }, poolExecutor);

CompletableFuture<Integer> combinedFutureObj = asyncTask1.thenCombine(asyncTask2, (Integer val1, String val2) -> val1 + val2);
```